

Requirements for BSVS

1. Functional Requirements (What the system must DO)

These describe system behaviors. For your project specifically:

Voter-Side Functions

- Voter must be able to register with verifiable identity credentials
- Voter must be able to authenticate before casting a ballot
- Voter must cast exactly one vote (double-voting must be impossible, not just discouraged)
- Voter must receive confirmation that their vote was recorded
- Voter must NOT be able to see how others voted

Admin-Side Functions

- Admin must be able to create an election (define candidates, open/close windows)
- Admin must be able to register eligible voters
- Admin must be able to close an election and trigger result computation
- Admin must NOT be able to alter votes after submission

Smart Contract Functions

- Contract must enforce all voting rules on-chain (not just in the frontend)
- Contract must emit events for every state-changing action
- Contract must expose a results function callable by anyone after election closes

Critical flag: Your current architecture has an unresolved question about who holds the admin key calling registerVoter. This is a **functional requirement gap**, not just a technical question. You cannot freeze the interface until this is answered. It belongs in this week's output.

2. Non-Functional Requirements (How well it must perform)

These are the ones students always write vaguely. Be specific or they're useless.

Category	What You Actually Need to Define
Security	What attack vectors must be prevented? (replay attacks, front-running, Sybil attacks, ballot stuffing)
Privacy	Is voter-ballot linkage traceable on-chain? If yes, is that acceptable for your capstone scope?

Category What You Actually Need to Define

Availability What's the uptime requirement during an election window?

Scalability How many voters does your system need to handle? (50? 500? 50,000? Pick a number.)

Auditability Who can audit results, and how?

Integrity What's the mechanism to detect/prevent vote tampering?

Brutal note on privacy: Ethereum is a public ledger. If you emit a `VoteCast(voterAddress, candidateId)` event, ballot secrecy is zero. You need to decide NOW — in Week 1 — whether your system anonymizes votes (commitment schemes, ZK-proofs, vote batching) or whether you're explicitly stating ballot secrecy is out of scope. Do not leave this undefined and discover it in Week 9.

3. Constraints (Hard boundaries the system must operate within)

These aren't negotiable — they're given to you by the environment:

- **Technology constraint:** Solidity on EVM-compatible chain
 - **Network constraint:** Ganache for local dev, public testnet for deployment — these behave differently in critical ways (gas costs, block times, account management)
 - **Timeline constraint:** 12 weeks, with interface freeze mandatory by end of Week 2
 - **Team constraint:** 4 people, parallel workstreams, no single point of failure
 - **Capstone constraint:** Does your institution require a specific deliverable format? A demo? A written report? Get this now, not Week 11.
 - **Identity verification constraint:** You cannot integrate a real government identity system in 12 weeks. You need to define your simulated identity boundary explicitly and document it as a known limitation — not hide it.
-

4. System Architecture Requirements (The structural decisions)

This is where most teams are too vague. You need concrete answers to each of these:

Blockchain Layer

- Which testnet? (Sepolia is the current standard — Ropsten is deprecated, Rinkeby is gone)
- What's your gas cost budget per transaction?
- Will you use upgradeable contracts or fixed-deploy? (For capstone: fixed-deploy is fine, but say so explicitly)

Smart Contract Layer

- Single contract or multiple? (VoterRegistry + Ballot + Results — or monolithic?)
- Who deploys? What's the deployment account? How is the private key managed?
- How are candidates represented? (string names? uint IDs? — this affects Garv's frontend and Anu's contract simultaneously)

Authentication Layer (Swati)

- What does "verified voter" mean in your system?
- What credential does Swati's module produce that gets passed to the smart contract?
- How does the contract validate that credential without a trusted oracle?

Frontend Layer (Garv)

- MetaMask wallet connection assumed?
- What happens if a voter doesn't have a wallet?
- Web-only or mobile considerations?